

Комментарии к коду Задачи 2 по ОММ.

По идее код программы подходит для всех задач с нулевой неоднородностью в правой части, нулевыми ГУ любого типа и ненулевым НУ. Для этого нужно:

1. Изменить границы отрезков по x , y , t на свои
2. Изменить НУ под свои
3. Изменить заполнение метода прогонки, учитывая вид ГУ
4. Изменить метод переменных направлений, учитывая ГУ

Подробнее про пункт 3.

Вот у нас есть метод прогонки:

Суть метода прогонки.

$$\begin{cases} A_n y_{n-1} - C_n y_n + B_n y_{n+1} = -F_n, & n=1 \dots N-1 \\ y_0 = \alpha_1 y_1 + \nu_1 \\ y_N = \alpha_N y_{N-1} + \nu_N \end{cases} \quad \begin{array}{l} \text{3-диагональная} \\ \text{система} \end{array}$$

мы хотим привести её к 2-диагональному виду, т.е.

$$\begin{cases} y_n = \kappa_n y_{n+1} + \rho_n, & n=0 \dots N-1 \\ y_N = \tilde{\rho}_N \end{cases}$$

$y_{n-1} = \kappa_n y_n + \rho_n = \kappa_n (\kappa_{n+1} y_{n+1} + \rho_{n+1}) + \rho_n = \kappa_n \kappa_{n+1} y_{n+1} + \kappa_n \rho_{n+1} + \rho_n$

подставим в исходную систему $\Rightarrow$ найдем:

$$\begin{aligned} y_{n+1} (A_n \kappa_n \kappa_{n+1} - C_n \kappa_{n+1} + B_n) + (A_n \kappa_n \rho_{n+1} + A_n \rho_n - C_n \rho_{n+1} + F_n) &= 0 \\ \Rightarrow \begin{cases} A_n \kappa_n \kappa_{n+1} - C_n \kappa_{n+1} + B_n = 0 \\ A_n \kappa_n \rho_{n+1} + A_n \rho_n - C_n \rho_{n+1} + F_n = 0 \end{cases} \Rightarrow \begin{cases} \kappa_{n+1} = \frac{B_n}{C_n - A_n \kappa_n} \\ \rho_{n+1} = \frac{F_n + A_n \rho_n}{C_n - A_n \kappa_n} \end{cases} \end{aligned}$$

прямой ход прогонки

$$\begin{cases} y_0 = \alpha_1 y_1 + \nu_1 \\ y_n = \kappa_n y_{n+1} + \rho_n \end{cases} \Rightarrow \begin{cases} \alpha_1 = \alpha_1 \\ \rho_1 = \nu_1 \end{cases}$$

найдем теперь y_n по уже известным κ и ρ :

$$\begin{cases} y_n = \alpha_1 y_{n-1} + \nu_1 & \text{Г.У. 3-диаг.} \\ y_{n-1} = \kappa_n y_n + \rho_n & \text{Г.У. прямого хода} \end{cases} \Rightarrow y_n = \frac{\nu_1 + \alpha_1 \rho_n}{1 - \alpha_n \alpha_1} = \tilde{\rho}_n$$

т.е., зная y_n , выражаем y_{n-1} : $y_{n-1} = \kappa_n y_n + \rho_n$

зная y_{n-1} , выражаем y_{n-2} : $y_{n-2} = \alpha_{n-1} y_{n-1} + \nu_{n-1}$

и т.д. до начала y_0 .

А вот его реализация для прогонки по x в коде программы:

```
38 #реализация метода переменных направлений
39 1 usage
40 def F1(n, m, j): #неоднородность в правой части для перехода j->j+1/2
41     return 0.5*gamma_y*(u[n, m-1, j-1]+u[n, m+1, j-1])+(1-gamma_y)*u[n, m, j-1]
42
43 1 usage
44 def solve_x(m, j): #метод прогонки по x
45     alpha = np.zeros(N)
46     alpha[1] = 0 #значение вписывается из ГУ по x
47     beta = np.zeros(N)
48     A = B = 1/2*gamma_x
49     C = 1 + gamma_x
50     for n in range(1, N-1): #прямой ход прогонки
51         F_instant = -F1(n, m, j)
52         alpha[n+1] = B/(C - A*alpha[n])
53         beta[n+1] = (F_instant - A*beta[n])/(A*alpha[n]-C)
54     u[N-1, m, j] = 0 #значение вписывается из ГУ по x
55     for n in range(N-1, 0, -1): #обратный ход прогонки
56         u[n-1, m, j] = alpha[n]*u[n, m, j] + beta[n]
```

Давайте посмотрим как его заполнять и как подгонять под другие задачи. Во-первых - элементы $\alpha[1]$ и $\beta[1]$ заполняются по граничному условию на начале интересующего нас отрезка, ведь:

$$\alpha_1 = \kappa_1, \quad \beta_1 = \nu_1, \quad y_0 = \kappa_1 * y_1 + \nu_1$$

Но по ГУ на отрезке x $u(x=0, \dots) = 0$, а значит $y_0 = 0$ и $\kappa_1 = \nu_1 = 0$, а стало быть и $\alpha_1 = \beta_1 = 0$. Для массива альфа это условие прописано явно в строке 44, но оба массива изначально инициализируются нулями, поэтому в целом можно и не прописывать.

Строки 46-51 нас не интересуют, они универсальны для метода прогонки. Следующая очень интересующая нас строка - это 52. В ней мы должны инициализировать y_N , чтобы реализовать обратный ход прогонки и вычислять $N-1$, $N-2$, ... значения. Но y_N - это граничное условие на конце интересующего

нас отрезка по x , то есть $u(x = 1, \dots)$. В моем коде ГУ на конце отрезка по x $[0, 1]$ соответствует $N - 1$ -й индекс трехмерного массива решения $u[n, m, j]$, поэтому:

$$u[N - 1, m, j] = 0$$

Строки 53-54 реализуют обратный ход прогонки и универсальны.

Иначе дело обстоит с прогонкой по y , ведь там стоит производная на правом конце. Поскольку производная в точке $y=2$ аппроксимируется выражением:

$$\frac{u[n, M - 1, j] - u[n, M - 2, j]}{h_y} = 0$$

То в качестве ГУ при $y=2$ получим:

$$u[n, M - 1, j] = u[n, M - 2, j]$$

Здесь, как и в случае с переменной x , концу отрезка $y=2$ соответствует индекс $M - 1$. Реализация метода прогонки по y в коде выглядит так:

```

1 usage
56 def F2(n, m, j):           #неоднородность в правой части для перехода j+1/2->j+1
57     return 0.5*gamma_x*(u[n-1, m, j-1]+u[n+1, m, j-1])+(1-gamma_x)*u[n, m, j-1]
58
1 usage
59 def solve_y(n, j):         #метод прогонки по y
60     alpha = np.zeros(M)
61     alpha[1] = 0           #значение вписывается из ГУ по y
62     beta = np.zeros(M)
63     A = B = 1/2*gamma_y
64     C = 1 + gamma_y
65     for m in range(1, M-1): #прямой ход прогонки
66         F_instant = -F2(n, m, j)
67         alpha[m+1] = B/(C - A*alpha[m])
68         beta[m+1] = (F_instant - A*beta[m])/(A*alpha[m]-C)
69     u[n, M-1, j] = beta[M-1]/(1-alpha[M-1]) #значение вписывается из ГУ по y
70     for m in range(M-1, 0, -1): #обратный ход прогонки
71         u[n, m-1, j] = alpha[m]*u[n, m, j]+beta[m]

```

Поскольку при $y=0$ имеем $u(\dots, y = 0, \dots) = 0$, то как и в прогонке по x получим $\alpha_1 = \beta_1 = 0$. То есть в строках 59-68 все аналогично прогонке по x . А вот в строке 69 нужно заполнить ГУ по y в точке $y=2$. Из общего алгоритма метода прогонки (вторая картинка) оно имеет вид:

$$y_N = \frac{\nu_2 + \kappa_2 * \beta_N}{1 - \alpha_N * \kappa_2}$$

При этом α_N и β_N нам известны из прямого хода прогонки. А согласно первой картинке общего алгоритма:

$$y_N = \kappa_2 * y_{N-1} + \nu_2$$

Но поскольку согласно ГУ $y_N = u[n, M - 1, j] = u[n, M - 2, j] = y_{N-1}$, то $\kappa_2 = 1$, $\nu_2 = 0$ (В общем алгоритме у нас был индекс N , но поскольку мы прогонку делаем по y , а в коде этой переменной соответствует индекс M , то ошибки в индексах нет. Индексу N из общего абстрактного метода соответствует индекс M из нашего кода.)

Короче, окончательно можно написать:

$$y_N = \frac{\beta_N}{1 - \alpha_N} \quad \text{или} \quad y_N = \frac{\beta[M - 1]}{1 - \alpha[M - 1]}$$

Итак, вы научились модифицировать метод прогонки под свои задачи.

Подробнее про пункт 4.

Метод переменных направлений выглядит так:

```

73     #собственно метод переменных направлений, заполнение массива решения
74     for j in range(1, 2*J, 2):
75         for m in range(1, M-1):
76             solve_x(m, j)
77         #граничные условия по x
78         u[0, :, :] = u[N-1, :, :] = 0
79         #граничные условия по y
80         u[:, 0, :] = 0
81         u[:, M-1, :] = u[:, M-2, :]
82         for n in range(1, N-1):
83             solve_y(n, j+1)

```

Понятное дело, что его нужно изменить в соответствии с ГУ для вашей конкретной задачи. В этой задаче имеем $u(\dots) = 0$ в точках $x=0$, $x=1$, $y=0$ и $\partial u / \partial y = 0$ в точке $y=2$. Именно эти ГУ и записаны.

Пару слов про погрешности.

Погрешность в этой задаче вычисляется как максимум модуля разности численного и аналитического решений в момент времени $t=0.01$, при этом в консоль выводятся значение погрешности и значение величины $h_x^2 + h_y + \tau^2$ на данной сетке (см. картинку).

```

117     #эта часть кода выводит величину h_x^2+h_y+tau^2 и максимум модуля ошибки для
118     #заданного в начале программы количества узлов сетки N, M, J
119     max_error = np.max(error)
120     accuracy = h_x**2+h_y+tau**2
121     print(accuracy)
122     print(max_error)
...
for n in range(0, N) > for m in range(0, M)

```

main x

```

"D:\VI семестр\ОММ\ОММ_Задание_2\ОММ_2_V1\.venv\Scripts\python.exe" "D:\VI семестр\ОММ\ОММ_Задание_2\ОММ_2_V1\main.
0.102501000000000001
0.02856437884653662

```

Для того чтобы построить график зависимости погрешности от $h_x^2 + h_y + \tau^2$ на разных сетках, нужно 4-5 раз запускать программу, вручную меняя значения разбиений сетки, а числа из консоли каждый раз записывать в Origin и уже там строить обычный график, как на праках. Не очень удобно, но работает. Ничего лучше у меня для вас нет.